

Assignment-12.3

T.Ramyasri
2303A52136

Batch 41

Task-1: Sorting Student Records for Placement Drive

PROMPT:

Write a Python script `placements_sort.py` that defines a `Student` dataclass with fields `name`, `roll`, and `cgpa`, generates datasets using familiar names like `Raju`, `Rahul`, `Rani`, `Priya`, etc., implements manual Quick Sort and Merge Sort, sorts students by CGPA descending with `name` and `roll` as tie-breakers, measures average runtimes for configurable dataset sizes and runs, validates against Python's `sorted()`, and prints clear sections showing correctly sorted student records, performance comparison with `speedup`, and the top 10 students, all via an `argparse` CLI with docstrings and type hints.

CODE:

```
Welcome X Lab-14.py X
Welcome py > main
1 #Write a Python script placements_sort.py that defines a Student dataclass with fields name, roll, a
2 from __future__ import annotations
3 import argparse
4 import random
5 import time
6 from dataclasses import dataclass
7 from typing import List, Callable
8 @dataclass
9 class Student:
10     """Dataclass representing a student."""
11     name: str
12     roll: int
13     cgpa: float
14 def generate_students(size: int) -> List[Student]:
15     """Generate a random dataset of students with familiar names."""
16     sample_names = [
17         "Raju", "Rahul", "Rani", "Priya", "Sita", "Gita", "Mohan",
18         "Arjun", "Anjali", "Sunita", "Vijay", "Kiran", "Asha",
19         "Deepak", "Neha", "Pooja", "Rajesh", "Meena", "Suresh", "Lakshmi"
20     ]
21     students = []
22     for i in range(size):
23         name = random.choice(sample_names)
24         roll = i + 1
25         cgpa = round(random.uniform(5.0, 10.0), 2)
26         students.append(Student(name, roll, cgpa))
27     return students
28 def compare_students(a: Student, b: Student) -> int:
29     """Comparison function for students: CGPA desc, then name asc, then roll asc."""
30     if a.cgpa != b.cgpa:
31         return -1 if a.cgpa > b.cgpa else 1
32     if a.name != b.name:
33         return -1 if a.name < b.name else 1
```

```

Welcome  Lab-14.py X
Lab-14.py > main
34     return -1 if a.roll < b.roll else 1 if a.roll > b.roll else 0
35 def quick_sort(arr: List[Student]) -> List[Student]:
36     """Quick Sort implementation."""
37     if len(arr) <= 1:
38         return arr
39     pivot = arr[len(arr) // 2]
40     left = [x for x in arr if compare_students(x, pivot) < 0]
41     middle = [x for x in arr if compare_students(x, pivot) == 0]
42     right = [x for x in arr if compare_students(x, pivot) > 0]
43     return quick_sort(left) + middle + quick_sort(right)
44 def merge_sort(arr: List[Student]) -> List[Student]:
45     """Merge Sort implementation."""
46     if len(arr) <= 1:
47         return arr
48     mid = len(arr) // 2
49     left = merge_sort(arr[:mid])
50     right = merge_sort(arr[mid:])
51     return merge(left, right)
52 def merge(left: List[Student], right: List[Student]) -> List[Student]:
53     """Helper function to merge two sorted lists."""
54     result = []
55     i = j = 0
56     while i < len(left) and j < len(right):
57         if compare_students(left[i], right[j]) <= 0:
58             result.append(left[i])
59             i += 1
60         else:
61             result.append(right[j])
62             j += 1
63     result.extend(left[i:])
64     result.extend(right[j:])
65     return result
66 def measure_runtime(sort_fn: Callable[[List[Student]], List[Student]], dataset: List[Student], runs:

```

```

Welcome  Lab-14.py X
Lab-14.py > main
66 def measure_runtime(sort_fn: Callable[[List[Student]], List[Student]], dataset: List[Student], runs:
67     """Measure average runtime of a sorting function."""
68     total_time = 0.0
69     for _ in range(runs):
70         data_copy = dataset[:]
71         start = time.perf_counter()
72         sorted_data = sort_fn(data_copy)
73         end = time.perf_counter()
74         total_time += (end - start)
75         # Validate correctness
76         assert sorted_data == sorted(dataset, key=lambda s: (-s.cgpa, s.name, s.roll))
77     return total_time / runs
78 def main() -> None:
79     parser = argparse.ArgumentParser(description="Compare Quick Sort and Merge Sort for student place
80     parser.add_argument("--size", type=int, default=1000, help="Number of students in dataset")
81     parser.add_argument("--runs", type=int, default=5, help="Number of runs for averaging runtime")
82     args = parser.parse_args()
83     dataset = generate_students(args.size)
84     quick_time = measure_runtime(quick_sort, dataset, args.runs)
85     merge_time = measure_runtime(merge_sort, dataset, args.runs)
86     speedup = merge_time / quick_time if quick_time > 0 else float("inf")
87     print("\n=====")
88     print(" Correctly Sorted Student Records ")
89     print("=====")
90     sorted_students = quick_sort(dataset)
91     for student in sorted_students[:20]: # show first 20 for verification
92         print(f"{student.name:<8} Roll:{student.roll:<4} CGPA:{student.cgpa:.2f}")
93     print("\n=====")
94     print(" Performance Comparison ")
95     print("=====")
96     print(f"Dataset size: {args.size}, Runs: {args.runs}")
97     print(f"Quick Sort: {quick_time:.6f} seconds (avg)")
98     print(f"Merge Sort: {merge_time:.6f} seconds (avg)")

```

```

Welcome  Lab-14.py X
Lab-14.py > main
78 def main() -> None:
98     print(f"Merge Sort: {merge_time:.6f} seconds (avg)")
99     print(f"Speedup (Merge/Quick): {speedup:.2f}x")
100    print("\n=====")
101    print(" Top 10 Students ")
102    print("=====")
103    for i, student in enumerate(sorted_students[:10], start=1):
104        print(f"{i:2d}. Name: {student.name}, Roll: {student.roll}, CGPA: {student.cgpa:.2f}")
105 if __name__ == "__main__":
106     main()

```

OUTPUT:

```
Correctly Sorted Student Records
=====
Meena      Roll:223   CGPA:10.00
Rani       Roll:770   CGPA:9.99
Sunita     Roll:358   CGPA:9.98
Suresh     Roll:590   CGPA:9.98
Arjun      Roll:378   CGPA:9.97
Asha       Roll:210   CGPA:9.97
Kiran      Roll:610   CGPA:9.97
Pooja      Roll:894   CGPA:9.97
Rajesh     Roll:462   CGPA:9.97
Sunita     Roll:508   CGPA:9.97
Neha       Roll:519   CGPA:9.96
Anjali     Roll:583   CGPA:9.95
Asha       Roll:172   CGPA:9.95
Sita       Roll:888   CGPA:9.95
Arjun      Roll:665   CGPA:9.94
Meena      Roll:429   CGPA:9.93
Neha       Roll:644   CGPA:9.93
Deepak     Roll:293   CGPA:9.91
Priya      Roll:367   CGPA:9.91
Priya      Roll:706   CGPA:9.91
=====
Dataset size: 1000, Runs: 5
Quick Sort: 0.006461 seconds (avg)
Merge Sort: 0.002273 seconds (avg)
Speedup (Merge/Quick): 0.35x
=====
Top 10 Students
=====
1. Name: Meena, Roll: 223, CGPA: 10.00
2. Name: Rani, Roll: 770, CGPA: 9.99
3. Name: Sunita, Roll: 358, CGPA: 9.98
4. Name: Suresh, Roll: 590, CGPA: 9.98
5. Name: Arjun, Roll: 378, CGPA: 9.97
6. Name: Asha, Roll: 210, CGPA: 9.97
7. Name: Kiran, Roll: 610, CGPA: 9.97
8. Name: Pooja, Roll: 894, CGPA: 9.97
9. Name: Rajesh, Roll: 462, CGPA: 9.97
10. Name: Sunita, Roll: 508, CGPA: 9.97
```

EXPLANATION: The code defines a Student dataclass and generates a dataset with familiar names like Raju, Rahul, Rani, Priya, etc. It implements manual Quick Sort and Merge Sort to order students by CGPA descending, with name and roll as tie-breakers. The program measures average runtimes for both algorithms over configurable dataset sizes and runs, validates correctness against Python's built-in sorted(), and prints three clear sections: a sample of correctly sorted records, a performance comparison with speedup, and the top 10 students. All functionality is accessible via an argparse CLI, with docstrings and type hints for clarity.

Task-2: Implementing Bubble Sort with AI Comments

PROMPT:

Write a Python script that implements Bubble Sort on a list of integers. Students should code the algorithm normally, and then ask AI to generate inline comments explaining the key logic (like swapping, passes, and termination). The AI should also provide a short time complexity analysis. The expected output is a Bubble Sort implementation.

CODE:

```
Lab-14.py > ...
109 #Write a Python script that implements Bubble Sort on a list of integers. Students should code the al
110 from typing import List
111 def bubble_sort(arr: List[int]) -> List[int]:
112     n = len(arr)
113     # Outer loop: controls the number of passes
114     for i in range(n):
115         # Inner loop: compares adjacent elements
116         for j in range(0, n - i - 1):
117             # If the current element is greater than the next, swap them
118             if arr[j] > arr[j + 1]:
119                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
120             # After each pass, the largest element among the unsorted part
121             # "bubbles up" to its correct position at the end
122         # When no swaps are needed, the list is sorted and the algorithm terminates
123     return arr
124 if __name__ == "__main__":
125     data = [64, 34, 25, 12, 22, 11, 90]
126     print("Original array:", data)
127     sorted_data = bubble_sort(data)
128     print("Sorted array:", sorted_data)
129     # Complexity Analysis:
130     # Bubble Sort has worst-case and average-case time complexity  $O(n^2)$ ,
131     # because it requires nested loops over the list.
132     # Best case is  $O(n)$  when the list is already sorted (with an optimized version).
133     # Space complexity is  $O(1)$ , since sorting is done in-place.
```

OUTPUT:

```
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]
```

EXPLANATION:

Bubble Sort works by repeatedly comparing adjacent elements in a list and swapping them if they are out of order, so that with each pass the largest unsorted element moves to its correct position at the end. The outer loop controls the number of passes, while the inner loop performs the comparisons and swaps. The process continues until no swaps are needed, meaning the list is sorted. Its worst-case and average-case time complexity is $O(n^2)$ due to the nested loops, while the best case can be $O(n)$ if optimized to stop early when the list is already sorted, and it uses only $O(1)$ extra space since sorting is done in place.

Task-3: Quick Sort and Merge Sort Comparison PROMPT:

Write a Python script that implements Quick Sort and Merge Sort using recursion. Provide partially completed functions and ask AI to fill in the missing recursive logic and add docstrings. Then compare both algorithms on random, sorted, and reverse-sorted lists. The expected output should include working implementations of Quick Sort and Merge Sort, along with AI-generated explanations of their average, best, and worst-case time complexities.

CODE:

```
Welcome  Lab-14.py X
Lab-14.py > ...
136 #Write a Python script that implements Quick Sort and Merge Sort using recursion. Provide partially
137 import random
138 import time
139 from typing import List
140 def quick_sort(arr: List[int]) -> List[int]:
141     """
142     Recursive Quick Sort implementation.
143     Splits the list around a pivot and recursively sorts sublists.
144     """
145     if len(arr) <= 1:
146         return arr
147     pivot = arr[len(arr) // 2]
148     left = [x for x in arr if x < pivot]
149     middle = [x for x in arr if x == pivot]
150     right = [x for x in arr if x > pivot]
151     return quick_sort(left) + middle + quick_sort(right)
152 def merge_sort(arr: List[int]) -> List[int]:
153     """
154     Recursive Merge Sort implementation.
155     Divides the list into halves, recursively sorts them,
156     and merges the sorted halves.
157     """
158     if len(arr) <= 1:
159         return arr
160     mid = len(arr) // 2
161     left = merge_sort(arr[:mid])
162     right = merge_sort(arr[mid:])
163     return merge(left, right)
164 def merge(left: List[int], right: List[int]) -> List[int]:
165     """Helper function to merge two sorted lists."""
166     result = []
167     i = j = 0
168     while i < len(left) and j < len(right):
```

```
Welcome  Lab-14.py X
Lab-14.py > ...
164 def merge(left: List[int], right: List[int]) -> List[int]:
169     if left[i] <= right[j]:
170         result.append(left[i])
171         i += 1
172     else:
173         result.append(right[j])
174         j += 1
175     result.extend(left[i:])
176     result.extend(right[j:])
177     return result
178 def measure_runtime(sort_fn: List[int], runs: int = 3) -> float:
179     """Measure average runtime of a sorting function."""
180     total = 0.0
181     for _ in range(runs):
182         data_copy = arr[:]
183         start = time.perf_counter()
184         sort_fn(data_copy)
185         end = time.perf_counter()
186         total += (end - start)
187     return total / runs
188 def main():
189     sizes = [1000]
190     test_cases = {
191         "Random": lambda n: [random.randint(0, 10000) for _ in range(n)],
192         "Sorted": lambda n: list(range(n)),
193         "Reverse-Sorted": lambda n: list(range(n, 0, -1)),
194     }
195     for size in sizes:
196         print(f"\n=== Dataset Size: {size} ===")
197         for case_name, generator in test_cases.items():
198             dataset = generator(size)
199             quick_time = measure_runtime(quick_sort, dataset)
200             merge_time = measure_runtime(merge_sort, dataset)
```

```
Welcome  Lab-14.py X
Lab-14.py > ...
188 def main():
201     print(f"\nCase: {case_name}")
202     print(f"Quick Sort: {quick_time:.6f} seconds (avg)")
203     print(f"Merge Sort: {merge_time:.6f} seconds (avg)")
204 if __name__ == "__main__":
205     main()
```

OUTPUT:

```
=== Dataset Size: 1000 ===

Case: Random
Quick Sort: 0.001073 seconds (avg)
Merge Sort: 0.001181 seconds (avg)

Case: Sorted
Quick Sort: 0.000676 seconds (avg)
Merge Sort: 0.000832 seconds (avg)

Case: Reverse-Sorted
Quick Sort: 0.000929 seconds (avg)
Merge Sort: 0.000956 seconds (avg)
```

EXPLANATION:

This script provides a clear comparison between Quick Sort and Merge Sort across different types of input data. It uses recursive implementations for both sorting algorithms, which are easier to understand and maintain. The performance measurement is done using the time module, and the results are averaged over multiple runs to ensure reliability. The use of lambda functions for generating test cases keeps the code concise and flexible. Overall, this script effectively demonstrates the differences in performance between the two sorting algorithms under various conditions.

Task-4: Real-Time Application – Inventory Management System

PROMPT:

Design a Python program for a retail store's inventory management system. The inventory contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Staff need to quickly search for products by ID or name, and sort products by price or quantity for stock analysis. Use AI to suggest the most efficient search and sort algorithms for this use case, implement them in Python, and justify the choices based on dataset size, update frequency, and performance requirements. The expected output should include a table mapping operation → recommended algorithm → justification, plus working Python functions for searching and sorting the inventory.

CODE:

```
Welcome  Lab-14.py X
Lab-14.py > main
201 #Design a Python program for a retail store's inventory management system. The inventory contains th
202 from dataclasses import dataclass
203 from typing import List, Optional
204 @dataclass
205 class Product:
206     product_id: int
207     name: str
208     price: float
209     quantity: int
210 # --- Search Algorithms ---
211 def binary_search(products: List[Product], target_id: int) -> Optional[Product]:
212     """Binary search for product by ID (requires sorted by ID)."""
213     low, high = 0, len(products) - 1
214     while low <= high:
215         mid = (low + high) // 2
216         if products[mid].product_id == target_id:
217             return products[mid]
218         elif products[mid].product_id < target_id:
219             low = mid + 1
220         else:
221             high = mid - 1
222     return None
223 def linear_search(products: List[Product], target_name: str) -> Optional[Product]:
224     """Linear search for product by name (works on unsorted data)."""
225     for product in products:
226         if product.name.lower() == target_name.lower():
227             return product
228     return None
229 # --- Sort Algorithms ---
230 def merge_sort(products: List[Product], key: str) -> List[Product]:
231     """Merge Sort implementation to sort products by a given key."""
232     if len(products) <= 1:
233         return products
```

```

Welcome  Lab-14.py X
Lab-14.py > main
def merge_sort(products: List[Product], key: str) -> List[Product]:
    mid = len(products) // 2
    left = merge_sort(products[:mid], key)
    right = merge_sort(products[mid:], key)
    return merge(left, right, key)
def merge(left: List[Product], right: List[Product], key: str) -> List[Product]:
    """Helper function to merge two sorted lists."""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if getattr(left[i], key) <= getattr(right[j], key):
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
# --- Demo and Justification Output ---
def print_algorithm_table():
    print("\n=== Recommended Algorithms ===")
    print(f"{'Operation':<25}{ 'Algorithm':<20}{ 'Justification':<20}")
    print("-" * 70)
    print(f"{'Search by ID':<25}{ 'Binary Search':<20}"
          f"{'Efficient O(log n) lookup on sorted IDs':<20}")
    print(f"{'Search by Name':<25}{ 'Linear Search':<20}"
          f"{'Names unsorted; O(n) acceptable for occasional lookups':<20}")
    print(f"{'Sort by Price/Quantity':<25}{ 'Merge Sort':<20}"
          f"{'Stable O(n log n) sort, good for large datasets':<20}")
def main():
    # Sample inventory
    inventory = [

```

```

Welcome  Lab-14.py X
Lab-14.py > main
def main():
    Product(101, "Apple", 50.0, 120),
    Product(205, "Banana", 20.0, 200),
    Product(150, "Orange", 30.0, 80),
    Product(300, "Mango", 100.0, 50),
    ]
    # Show recommended algorithms
    print_algorithm_table()
    # Search examples
    print("\n=== Search Examples ===")
    sorted_by_id = merge_sort(inventory, "product_id")
    found = binary_search(sorted_by_id, 150)
    print("Search by ID 150:", found)
    found_name = linear_search(inventory, "Mango")
    print("Search by Name 'Mango':", found_name)
    # Sort examples
    print("\n=== Sort Examples ===")
    sorted_by_price = merge_sort(inventory, "price")
    print("Sorted by Price:", sorted_by_price)
    sorted_by_quantity = merge_sort(inventory, "quantity")
    print("Sorted by Quantity:", sorted_by_quantity)
if __name__ == "__main__":
    main()

```

OUTPUT:

```

Operation          Algorithm          Justification
-----
Search by ID       Binary Search      Efficient O(log n) l
lookup on sorted IDs
Search by Name     Linear Search      Names unsorted; O(n)
acceptable for occasional lookups
Sort by Price/Quantity Merge Sort         Stable O(n log n) so
rt, good for large datasets

=== Search Examples ===
Search by ID 150: Product(product_id=150, name='Orange', price=30
.0, quantity=80)
Search by Name 'Mango': Product(product_id=300, name='Mango', pri
ce=100.0, quantity=50)

=== Sort Examples ===
Sorted by Price: [Product(product_id=205, name='Banana', price=20
.0, quantity=200), Product(product_id=150, name='Orange', price=3
0.0, quantity=80), Product(product_id=101, name='Apple', price=50
.0, quantity=120), Product(product_id=300, name='Mango', price=10
0.0, quantity=50)]
Sorted by Quantity: [Product(product_id=300, name='Mango', price=
100.0, quantity=50), Product(product_id=150, name='Orange', price
=30.0, quantity=80), Product(product_id=101, name='Apple', price=
50.0, quantity=120), Product(product_id=205, name='Banana', price
=20.0, quantity=200)]

```

EXPLANATION:

The inventory management system code efficiently handles searching and sorting for thousands of products. For product ID lookups, Binary Search is used because IDs are numeric and can be kept sorted, giving fast $O(\log n)$ performance. For product name lookups, Linear Search is chosen since names are not guaranteed to be sorted, and occasional $O(n)$ scans are acceptable. For sorting by price or quantity, Merge Sort is implemented because it is stable, consistently runs in $O(n \log n)$, and works well for large datasets. The program outputs a clear table mapping each operation to its recommended algorithm with justification, and provides working Python functions (`binary_search`, `linear_search`, `merge_sort`) that demonstrate these operations on sample inventory data. This design balances speed, scalability, and practicality for real-time inventory management.

Task-5: Real-Time Stock Data Sorting & Searching

PROMPT:

This program simulates stock price data and provides a simple menu-driven analysis tool. It generates random stocks with symbols, opening and closing prices, and calculates daily percentage change. Heap Sort is used to rank stocks efficiently by gain or loss, while a Hash Map (dictionary) enables instant $O(1)$ lookups by symbol. The menu allows users to view the top five stocks, search for a specific symbol, compare Heap Sort performance against Python's built-in `sorted()` function, or exit. This design combines algorithmic efficiency with practical usability, making it suitable for real-time stock analysis.

CODE:

```
Welcome  Lab-14.py X
Lab-14.py > ...
290 #This program simulates stock price data and provides a simple menu-driven analysis tool. It generates
291 import random
292 import time
293 def generate_stocks(n):
294     symbols=["TCS","INFY","HDFC","RELIANCE","WIPRO","SBIN","ITC","ADANI","AXIS","ICICI"]
295     stocks=[]
296     for i in range(n):
297         symbol=random.choice(symbols)+str(i)
298         open_price=random.randint(100,5000)
299         close_price=open_price+random.randint(-200,200)
300         percent_change=((close_price-open_price)/open_price)*100
301         stocks.append({"symbol":symbol,"open":open_price,"close":close_price,"change":round(percent_c
302     return stocks
303 def heapify(arr,n,i):
304     largest=i
305     left=2*i+1
306     right=2*i+2
307     if left<n and arr[left]["change">arr[largest]["change"]:
308         largest=left
309     if right<n and arr[right]["change">arr[largest]["change"]:
310         largest=right
311     if largest!=i:
312         arr[i],arr[largest]=arr[largest],arr[i]
313         heapify(arr,n,largest)
314 def heap_sort(arr):
315     n=len(arr)
316     for i in range(n//2-1,-1,-1):
317         heapify(arr,n,i)
318     for i in range(n-1,0,-1):
319         arr[i],arr[0]=arr[0],arr[i]
320         heapify(arr,i,0)
321     return arr[::-1]
322 def build_hash_map(stocks):
```



```

323         return {stock["symbol"]:stock for stock in stocks}
324 def search_stock(stock_map,symbol):
325     return stock_map.get(symbol,"Stock not found")
326 # Generate data
327 n=1000
328 stocks=generate_stocks(n)
329 stock_map=build_hash_map(stocks)
330 sorted_heap=heap_sort(stocks.copy())
331 while True:
332     print("\n--- Stock Analysis Menu ---")
333     print("1. Show Top 5 Stocks")
334     print("2. Search Stock by Symbol")
335     print("3. Performance Comparison")
336     print("4. Exit")
337     choice=input("Enter your choice: ")
338     if choice=="1":
339         print("\nTop 5 Stocks by % Gain/Loss:\n")
340         for i in range(5):
341             s=sorted_heap[i]
342             print(f"{i+1}. {s['symbol']} | Open:{s['open']} | Close:{s['close']} | Change:{s['change']}")
343     elif choice=="2":
344         symbol=input("Enter Stock Symbol: ")
345         result=search_stock(stock_map,symbol)
346         print("Search Result:",result)
347     elif choice=="3":
348         start=time.perf_counter()
349         heap_sort(stocks.copy())
350         heap_time=time.perf_counter()-start
351         start=time.perf_counter()
352         sorted(stocks,key=lambda x:x["change"],reverse=True)
353         builtin_time=time.perf_counter()-start
354         print("\nPerformance Comparison:")
355         print("Heap Sort Time:",round(heap_time,6),"seconds")
356         print("Built-in sorted() Time:",round(builtin_time,6),"seconds")
357     else:
358         break
359     print("\n--- Stock Analysis Menu ---")

```

```

355     print("Heap Sort Time:",round(heap_time,6),"seconds")
356     print("Built-in sorted() Time:",round(builtin_time,6),"seconds")
357 elif choice=="4":
358     print("Exiting program...")
359     break
360 else:
361     print("Invalid choice! Try again.")

```

OUTPUT:

```

--- Stock Analysis Menu ---
1. Show Top 5 Stocks
2. Search Stock by Symbol
3. Performance Comparison
4. Exit
Enter your choice: 1

Top 5 Stocks by % Gain/Loss:

1. RELIANCE979 | Open:131 | Close:305 | Change:132.82%
2. ITC707 | Open:129 | Close:280 | Change:117.05%
3. INFY816 | Open:135 | Close:263 | Change:94.81%
4. TCS371 | Open:148 | Close:279 | Change:88.51%
5. ITC677 | Open:194 | Close:363 | Change:87.11%

--- Stock Analysis Menu ---
1. Show Top 5 Stocks
2. Search Stock by Symbol
3. Performance Comparison
4. Exit
Enter your choice: 4
Exiting program...

```

EXPLANATION:

This code simulates stock data and provides a menu-driven analysis tool. It generates random stocks with opening and closing prices, calculates percentage change, and ranks them using Heap Sort for efficient $O(n \log n)$ sorting. Searching is optimized with a Hash Map (dictionary), allowing instant $O(1)$ lookups by symbol. The menu lets users view the top five stocks, search by symbol, compare Heap Sort against Python's built-in `sorted()` function, or exit. The performance comparison shows that while built-ins are faster due to internal optimizations, Heap Sort demonstrates algorithmic efficiency and control, making the program practical for real-time stock analysis.